

Innføring i Opreativ Systemer, Obligatorisk oppgave 4: Prosesser og systemkall i Linux

Andreas Isaksen

September 20, 2024

Oppgave 1

Følgende C-program er gitt:

```
#include <stdio.h>
#include <sys/types.h>
#include <unistd.h>

int main()
{
    pid_t child1, child2;

    child1 = fork();
    child2 = fork();

    printf("Jeg er prosess %d, min forelder er %d\n",
           getpid(), getppid());
    sleep(1);
}
```

a) Kompiler og kjør programmet. Vis hvordan utskriften fra programmet ser ut.

Svar:

```
[aragnaroth@archlinux oblig4]$ cc -o oppg1 oppg1.c
[aragnaroth@archlinux oblig4]$ ./oppg1 &
[1] 159949
[aragnaroth@archlinux oblig4]$ Jeg er prosess 159949,
    min forelder er 57910
Jeg er prosess 159951, min forelder er 159949
Jeg er prosess 159950, min forelder er 159949
Jeg er prosess 159952, min forelder er 159950
```

Listing 1: shell terminal

b) Lag en figur som viser prosessreet for de fem prosessene som er aktive når dette programmet kjøres, med shellet som startet programmet øverst i treet. Bruk prosess-ID'ene fra utskriften i oppgave a ovenfor i figuren din.

Svar:

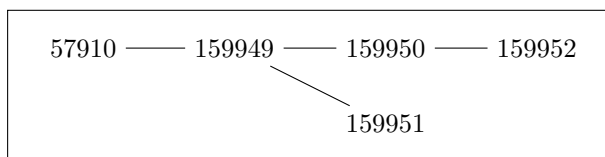


Figure 1: pstree av prosess 57910

c) Gi en kort forklaring på hvorfor det lages tre nye prosesser når dette programmet kjøres, selv om det bare er to kall på `fork()`.

Svar:

Kort forklart så kommer dette av at aktive prosesser i programmet vil doble seg ved denne typen bruk av `fork()`. `bash(57910)` starter programmet (oppg1) og vi får prosess 159949. Denne prosessen "forkes" og lager barneprosess 159950. Begge disse prosessene kjører gjennom neste `fork()` som gjør at foreldreprosessen får enda et barn (159951) og 159950 får et barn (159952) som blir barnebarnet til 159949.

I koden gitt ovenfor, "sover" alle prosessene i ett sekund (nederste kodelinje) før de terminerer. Nedenfor er en litt omskrevet versjon av koden, der bare de nye prosessene som lages med `fork()` sover:

```
#include <stdio.h>
#include <sys/types.h>
#include <unistd.h>

int main()
{
    pid_t child1, child2;

    child1 = fork();
    if (child1 == 0)
        sleep(1);

    child2 = fork();
```

```
if (child2 == 0)
    sleep(1);

printf("Jeg er prosess %d, min forelder er %d\n",
       getpid(), getppid());
}
```

d) Kompiler og kjør også dette programmet. Rediriger utskriften til en fil ved å bruke spesialtegnet '>' i shellet (se f.eks. oblig 1 for en forklaring av redigering av I/O i Linux). Inkluder denne filen/utskriften fra programmet i besvarelsen din.

Du vil se at forelder-barn forholdet mellom prosessene nå har forandret seg. Gi en forklaring på hva som skjer. Hint: Se avsnitt 1.2 ovenfor...

Svar:

```
[aragnaro@archlinux oblig4]$ cc -o oppg1d oppg1d.c
[aragnaro@archlinux oblig4]$ ./oppg1d > oppg1dOut.txt
```

Listing 2: shell terminal

```
Jeg er prosess 297977, min forelder er 57910
Jeg er prosess 297979, min forelder er 1
Jeg er prosess 297978, min forelder er 1
Jeg er prosess 297987, min forelder er 1
```

Listing 3: oppg1dOut.txt

Her forekommer tilnærmet det samme som i C-koden til oppgave 1 a). Den store forskjellen her er at de prosesser som faller utenfor parametrene til oppført if-statement ikke vil ta en pause(sleep) før de gjør seg ferdige. Dette medfører at foreldreprosesser avslutter seg selv før barneprosessen får gjort seg ferdig. Når dette skjer og foreldreprosessen terminerer seg selv, så vil barneprosessen bli adoptert av init/systemd prosessen som har PID 1. Når disse prosessene kjører ferdig vil de referere til PID 1 som nå er forelder til prosessen, da de har blitt adoptert før terminering.

Oppgave 2

Følgende C-program er gitt:

```
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int main()
{
    int status = 0;
    pid_t child = fork();

    if (child == 0)
    {
        printf("Jeg er barneprosessen med PID %d\n",
            getpid());
        sleep(1);
        return 42;
    }
    else if (child > 0)
    {
        wait(&status);
        printf("Jeg er forelderprosessen med PID %d\n",
            getpid());
        printf("Barneprosessen %d har terminert med
            returstatus lik %d\n",
            child, WEXITSTATUS(status));
    }
}
```

a) Kompiler og kjør programmet. Inkluder utskriften fra programmet i besvarelsen din.

Svar:

```
[aragnaroth@archlinux oblig4]$ cc -o oppg2 oppg2.c
[aragnaroth@archlinux oblig4]$ ./oppg2
Jeg er barneprosessen med PID 311878
Jeg er forelderprosessen med PID 311877
Barneprosessen 311878 har terminert med returstatus
lik 42
```

Listing 4: Shell Terminal

b) Ta deretter vekk kallet på wait() i koden, kompiler og kjør programmet på

nytt. Inkluder også denne utskriften fra programmet i besvarelsen din. Beskriv hva som har endret seg i utskriften i forhold til i oppgave a ovenfor. Gi en kort forklaring på hvorfor dette skjer.

Legg forøvrig merke til bruken av `WEXITSTATUS` for å hente ut korrekt returverdi fra barneprosessen i koden i denne oppgaven. Dette er en av flere såkalte macroer, som er definert i headerfilen `sys/wait.h`. Se f.eks. denne siden fra GNU om "Process Completion Status". for mer informasjon.

Svar:

```
[aragnaro@archlinux oblig4]$ cc -o oppg2 oppg2.c
[aragnaro@archlinux oblig4]$ ./oppg2
Jeg er forelderprosessen med PID 313559
Jeg er barneprosessen med PID 313560
Barneprosessen 313560 har terminert med returstatus
lik 0
```

Listing 5: Shell Terminal

Her forekommet mye av det samme, men siden foreldreprosessen nå ikke får noen beskjed om å vente på at barneprosessen skal fullføre før foreldreprosessen avslutter, så prøver foreldreprosessen å hente ut status fra barneprosessen. Siden barneprosessen ikke har returnert noen status enda, vil dette kunne føre til uforutsigbare resultater for foreldreprosessen når den kjører `WEXITSTATUS(status)`. I dette tilfellet vil den hente ut en standard verdi (0) og returnere dette istedet for 42 som er den verdien barneprosessen faktisk returnerer.

Oppgave 3

Når en prosess terminerer, vil Linux vanligvis ta ut prosessen fra prosesstabellen, slik at den fjernes helt fra systemet. Prosessens PID og entry i prosesstabellen kan da gjenbrukes. Noen ganger vil prosessen ikke kunne fjernes fra tabellen, selv om den er ferdig med å eksekvere all sin egen kode og derfor ikke kan "vekkes" igjen. Prosessen blir da en såkalt zombie-prosess, med status lik 'Z' i utskrift fra f.eks. kommandoen ps.

Det er oftest barneprosesser som blir zombier, når de terminerer før sin egen forelderprosess. Barneprosessen beholdes da i prosesstabellen selv om den er ferdig, fordi forelderprosessen kan trenge å hente ut barneprosessens returverdi/exit status. For at zombien skal kunne fjernes helt, må forelderprosessen enten selv terminere, eller gjøre et kall på wait() eller waitpid() som leser returverdien fra barnet.

Nedenfor er et lite C-program som bruker fork() til å lage en barneprosess, som deretter blir til en zombie. Dette gjøres ved å la forelderprosessen kjøre en evig løkke som kaller sleep, mens barneprosessen bare terminerer med en melding rett etter at den er opprettet:

```
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>

int main()
{
    int status = 0;
    pid_t child = fork();

    if (child == 0)
    {
        printf("Barneprosessen med PID %d avslutter\n",
            getpid());
        return 0;
    }
    else
    {
        printf("Forelderprosessen med PID %d starter en
            evig lokke\n",
            getpid());
        while (1)
            sleep(10);
    }
}
```

a) Kompiler koden ovenfor, la den eksekverbare filen få navnet oppgave_3. Kjør deretter programmet som en bakgrunnsprosess med kommandoen:

```
./oppgave_3 &
```

Du får da tilbake kommandopromptet og kan fortsette å gi kommandoer til shellet mens programmet oppgave_3 kjører i bakgrunnen. Gi nå følgende kommando:

```
ps -l
```

Du vil da se at en av prosessene har status 'Z'. Dette er barneprosessen fra programmet ovenfor, som fortsatt ligger i prosesstabellen som en zombie. Forelderprosessen som kjører i evig løkke vil også skrives ut av ps -l .

Klipp-og-lim eller ta et skjermdump av utskriften fra ps -l, og inkluder dette i besvarelsen din.

Svar:

```
[aragnaroth@archlinux oblig4]$ cc -o oppgave_3
oppgave_3.c
[aragnaroth@archlinux oblig4]$ ./oppgave_3 &
[1] 347265
[aragnaroth@archlinux oblig4]$ Forelderprosessen med
PID 347265 starter en evig lokke
Barneprosessen med PID 347266 avslutter
ps -l
F S  UID      PID      PPID    C PRI   NI  ADDR  SZ  WCHAN
TTY          TIME CMD
0 S  1000    346930   346929   0  80    0   -    1962 do_wai
pts/0        00:00:00 bash
0 S  1000    347265   346930   0  80    0   -     635 hrtime
pts/0        00:00:00 oppgave_
1 Z  1000    347266   347265   0  80    0   -      0 -
pts/0        00:00:00 oppgave_
0 R  1000    347316   346930  99  80    0   -    2136 -
pts/0        00:00:00 ps
```

Listing 6: Shell Terminal

b) Hvilken Linux-kommando kan du nå gi fra shellet for å bli kvitt zombieprosessen?

Svar:

Om det kun er ønskelig å stoppe zombieprosessen, men ikke stoppe loopen til foreldreprosessen, så kan man bruke *kill* kommandoen på barneprosessens PID. For dette tilfellet ville dette blitt: "kill 347266". Eller så kan man terminere foreldre prosessen, som også ville gitt zombieprosessen mulighet til å avslutte vanlig.

1 Oppgave 4

Skriv et C-program som gjør følgende:

- Starter en barneprosess med fork().
- Deretter skal forelderprosessen "sove" i 5 sekunder.
- Barneprosessen skal kalle execvp() slik at den bytter ut seg selv med en ny prosess som kjører Linux-kommandoen ping vg.no
- Når forelderprosessen "våkner opp" etter 5 sekunder, skal den terminere barneprosessen med et kall på kill().

Du kan selv velge om du vil sende barneprosessen et SIGINT- eller SIGKILL-signal, prøv gjerne begge deler og se forskjellen i hvordan ping-prosessen terminerer. Legg gjerne inn en kommentar i koden din som forklarer forskjellen.

Svar: Fil "oppgave_4.c" er vedlagt.

```
[aragnaroth@archlinux oblig4]$ cc -o oppgave_4
oppgave_4.c
[aragnaroth@archlinux oblig4]$ ./oppgave_4
Forelder (PID 382051) tar seg 5 mens barna (PID
382052) leker.
PING www.vg.no (195.88.55.16) 56(84) bytes of data.
64 bytes from www.vg.no (195.88.55.16): icmp_seq=1 ttl
=242 time=69.9 ms
64 bytes from www.vg.no (195.88.55.16): icmp_seq=2 ttl
=243 time=92.4 ms
64 bytes from www.vg.no (195.88.55.16): icmp_seq=3 ttl
=245 time=115 ms
64 bytes from www.vg.no (195.88.55.16): icmp_seq=4 ttl
=243 time=35.2 ms
64 bytes from www.vg.no (195.88.55.16): icmp_seq=5 ttl
=245 time=58.0 ms
Forelder (PID 382051) braavaakner og tar livet av
barnet (PID 382052) sitt...
Alt er stille naa. Barnet (PID 382052) er ikke lenger
med oss...
```

Listing 7: Shell Terminal